



# Lesson 3: Regression

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi  
dell'Aquila

# Before we start

- Exercise 2, which we discuss now
- The pipeline you built is the one we fit models into from now on

# Today: the first real model

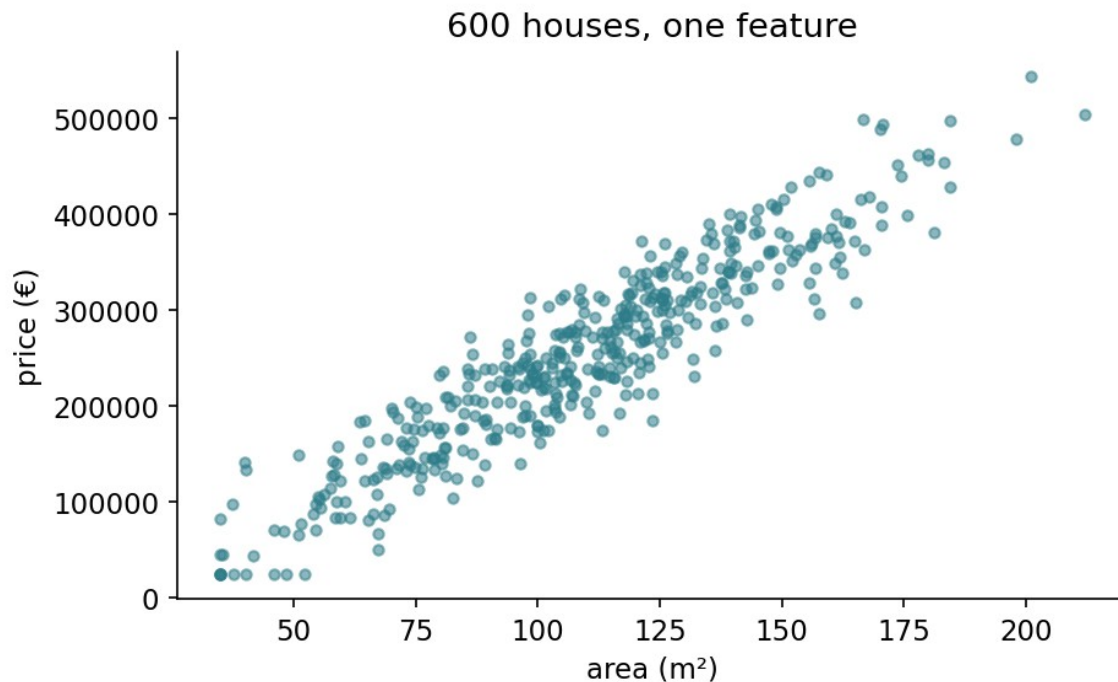
- The model and what it claims
- The exact solution, and why it sometimes fails
- Gradient descent
- Curves, and the price of flexibility
- Ridge and Lasso

# One dataset, all lesson

600 houses. Six measurements. A price.

**And we know the coefficients that generated it.**

# What we are trying to predict



# What a linear model claims

Each feature contributes a fixed amount per unit, and the contributions add up.

- Every square metre: the same 2,400 €
- Every kilometre out: the same –6,500 €

# Fitting means minimising something

We need one number saying how badly a candidate model is doing.

Adding up the errors fails:  $+50,000$  and  $-50,000$  cancel to zero.

# Why squared, not absolute

- **Differentiable everywhere:** no corner at zero
- **Punishes one large error more** than several small ones
- Under Gaussian noise, it **is** maximum likelihood



# The cost function: mean squared error

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

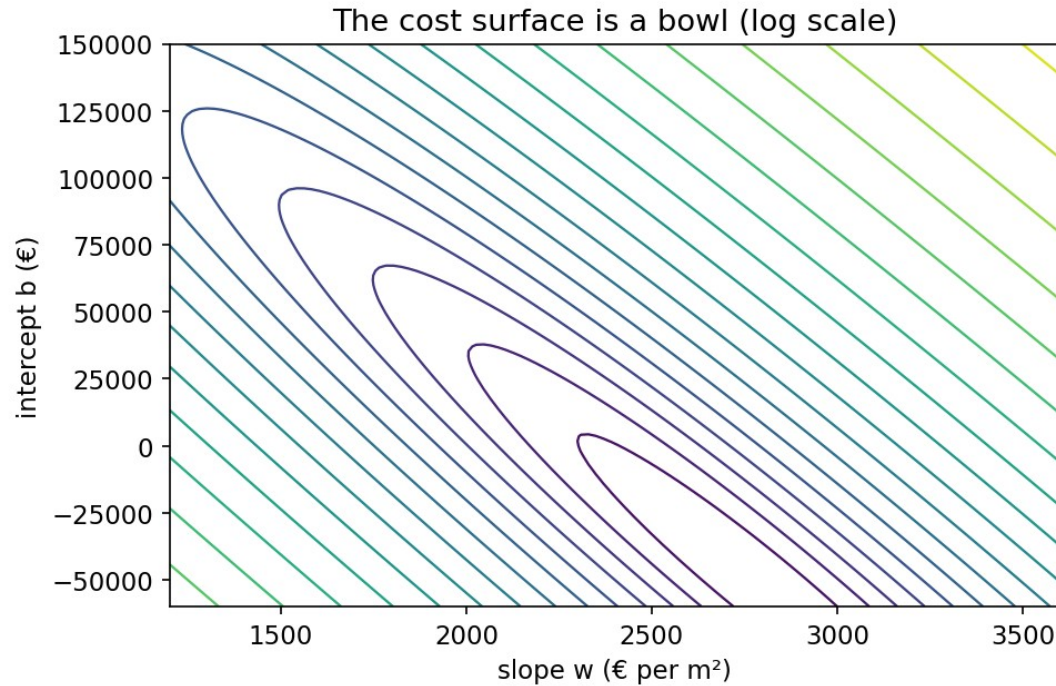
Trying  $\hat{y} = 2,400 \cdot \text{area} + 45,000$

| Area (m <sup>2</sup> ) | True (€) | Predicted (€) | Error $\hat{y} - y$ | Error <sup>2</sup> |
|------------------------|----------|---------------|---------------------|--------------------|
| 80                     | 240,000  | 237,000       | -3,000              | $9.0 \times 10^6$  |
| 120                    | 320,000  | 333,000       | +13,000             | $1.69 \times 10^8$ |
| 200                    | 540,000  | 525,000       | -15,000             | $2.25 \times 10^8$ |

# Squared error spends its attention on the worst house

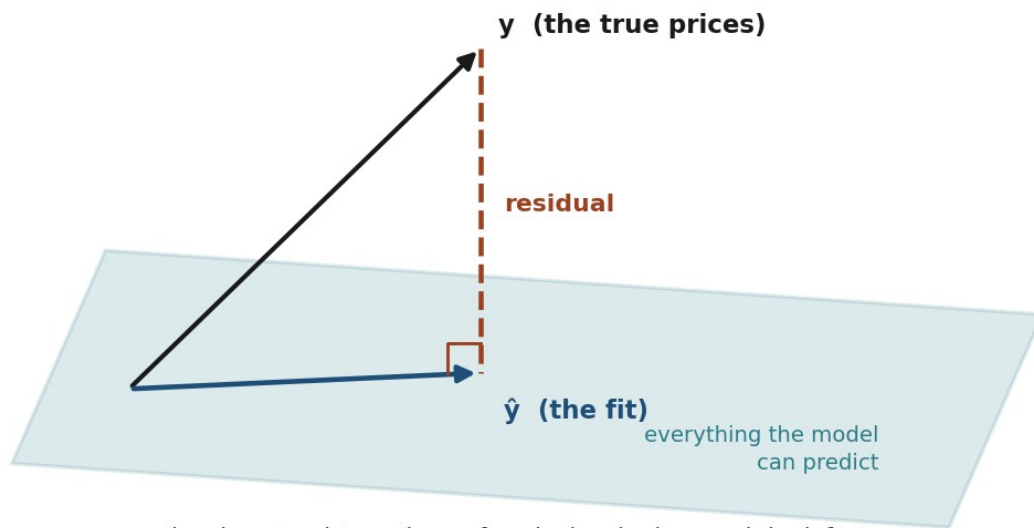
- The 3,000 € error contributes about **2%** of the total
- The two large errors contribute the other **98%**
- Halving the 15,000 € error cuts the cost by **42%**

# The cost is a bowl



# The picture behind the exact solution

Least squares is a projection



the closest point on the surface is the shadow, and the leftover error is perpendicular, or you could have done better

# The normal equation

$$\theta = (X^T X)^{-1} X^T y$$

# The same equation, by hand

| Quantity         | Value                                          |
|------------------|------------------------------------------------|
| the three houses | 80, 120, 200 m <sup>2</sup> → 240k, 320k, 540k |
| $X^T X$          | rows (3, 400) and (400, 60,800)                |
| $X^T y$          | (1,100, 165,600)                               |
| determinant      | 22,400                                         |
| <b>slope w</b>   | <b>2,536 €/m<sup>2</sup></b> (truth 2,400)     |
| intercept b      | 28,600 €                                       |

# And it really is the minimum

- The second derivative is  $X^T X$ , never negative in any direction
- So the cost is **convex**: one bottom, no local traps
- Any stationary point is *the* answer, not *an* answer



# So why learn anything else?

- $(X^T X)^{-1}$  costs about  $n^3$  operations
- Two columns carrying one fact → **no inverse exists**
- Nearly-redundant columns → an inverse that is useless

# How stretched is the valley?

The **condition number** compares the steepest direction with the shallowest.

- Housing features as recorded: **285**
- The same features, standardised: **3.4**
- Add `area_sqft` beside `area_sqm`: **2,286**

# The picture behind gradient descent

You are on a hillside in fog.

You cannot see the valley, but you can feel which way the ground slopes.

# The update rule

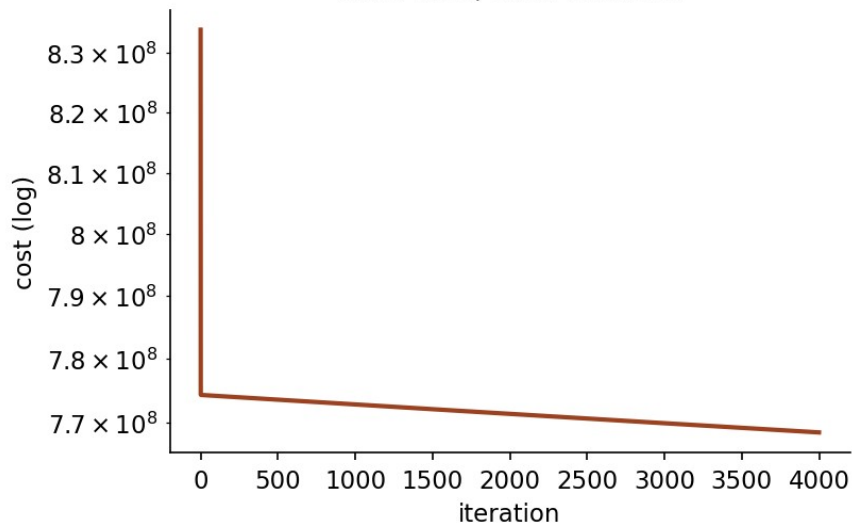
$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}, \quad \frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_i (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

# One step, from $w = 0$ and $b = 0$

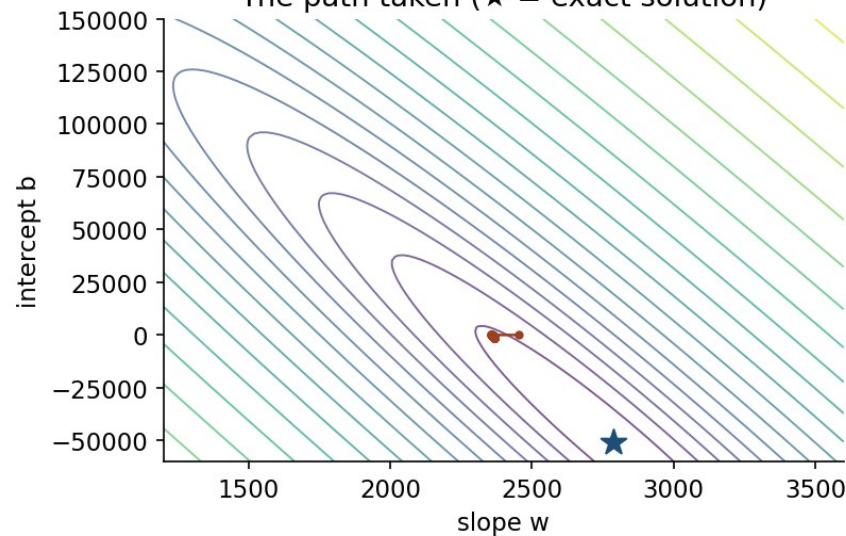
| $\alpha = 10^{-5}$ | Gradient | After one step |
|--------------------|----------|----------------|
| slope $w$          | -63,600  | 0.636          |
| intercept $b$      | -390     | 0.0039         |

# The path it takes

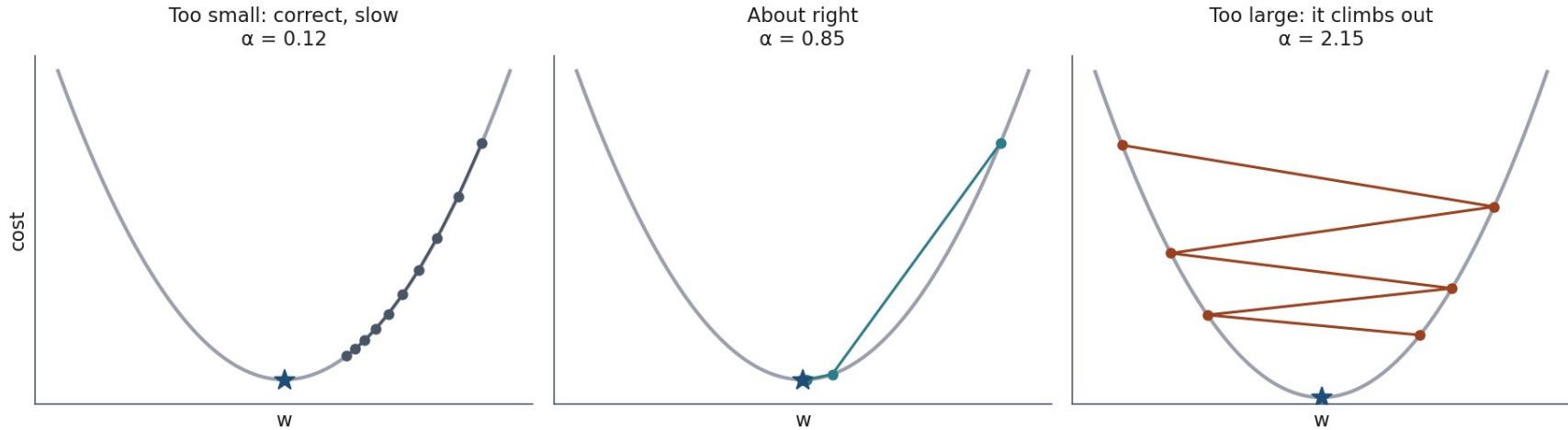
Cost falls, then flattens



The path taken (★ = exact solution)



# Three learning rates



# Notebook 1, live

The model, the cost, the exact solution, the iterative one: from scratch.



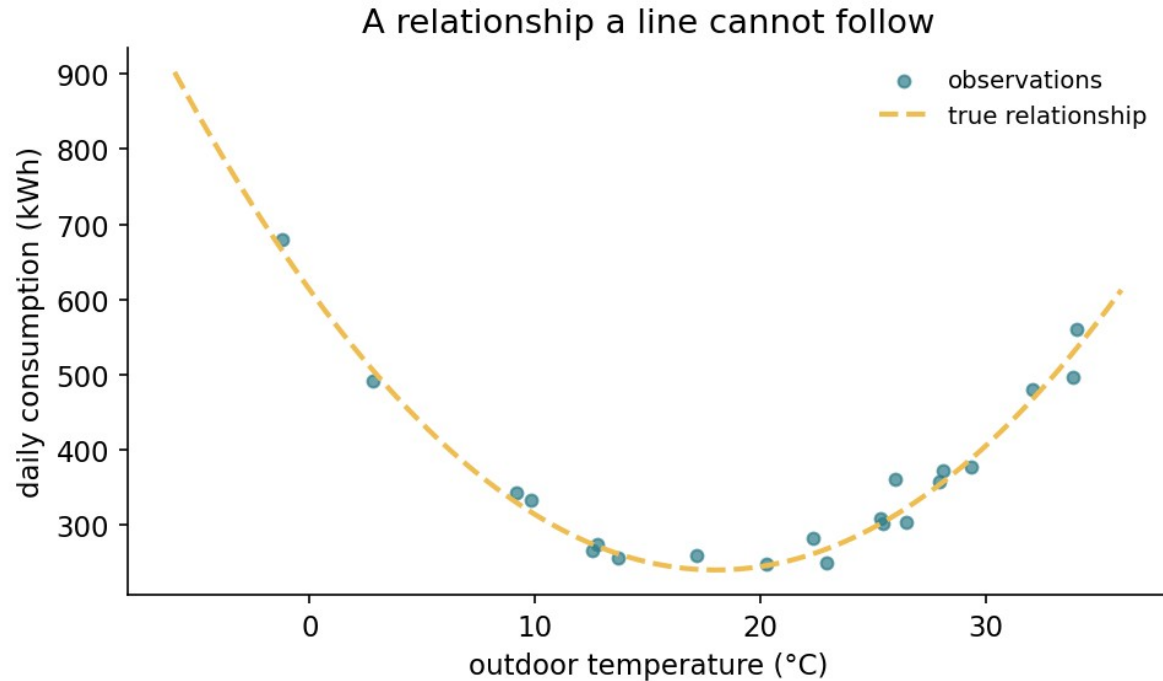
# Break

# What if the relationship bends?

Energy consumption against temperature: heating in the cold, cooling in the heat.

No straight line follows that.

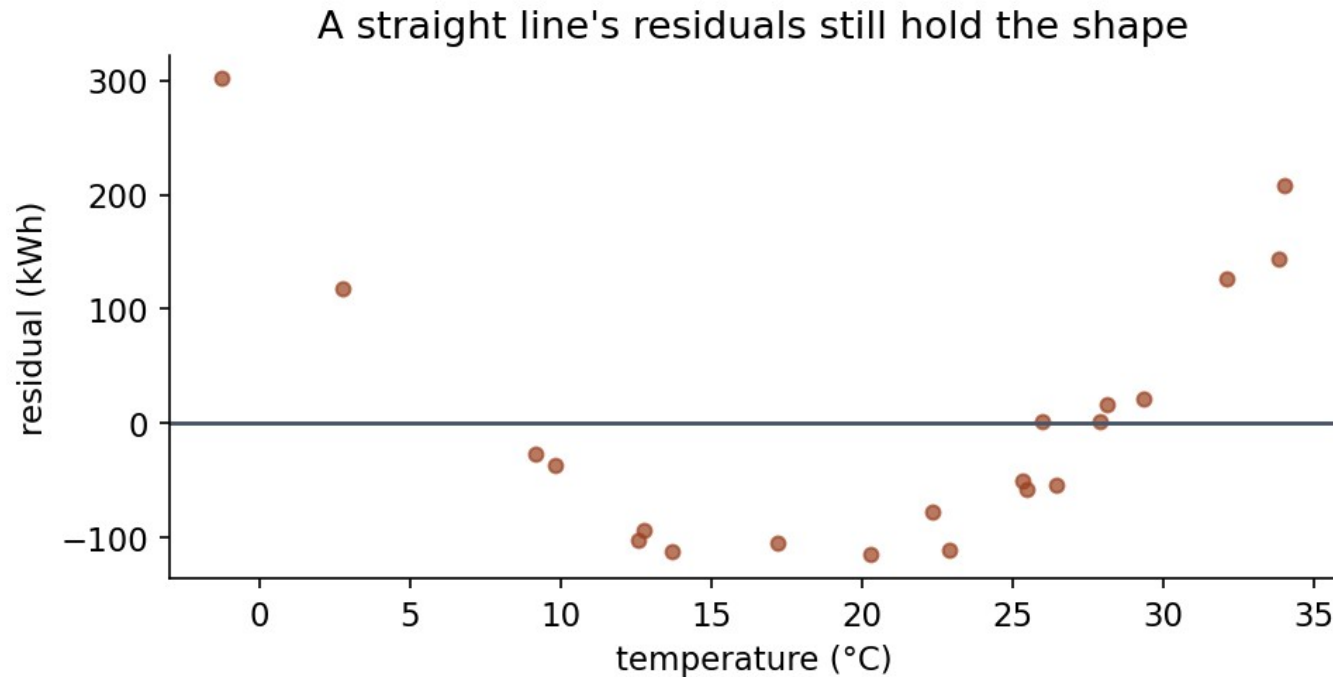
# A relationship a line cannot follow



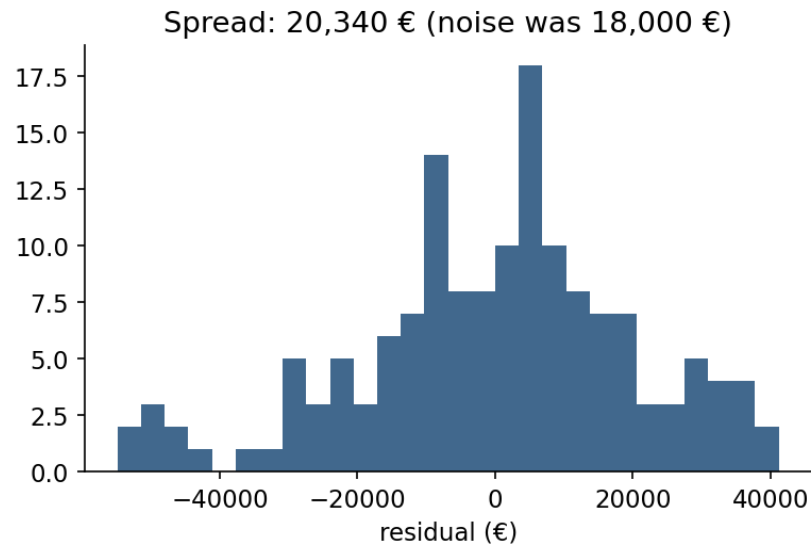
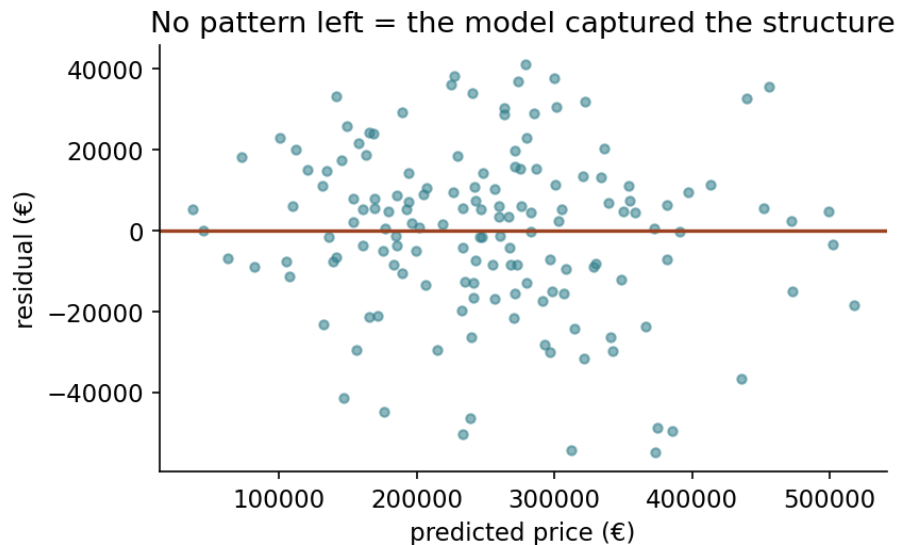
“Linear” means linear in the coefficients

$$\hat{y} = w_1 t + w_2 t^2 + b$$

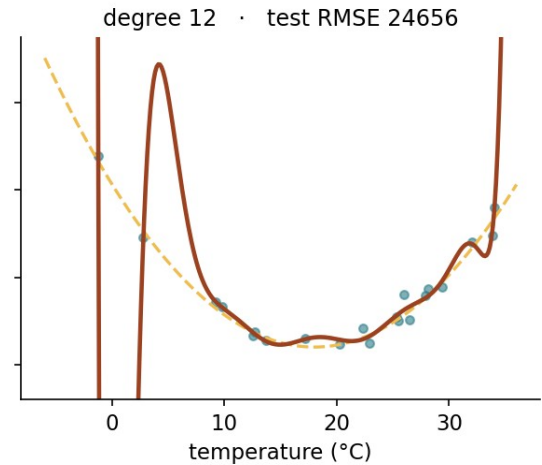
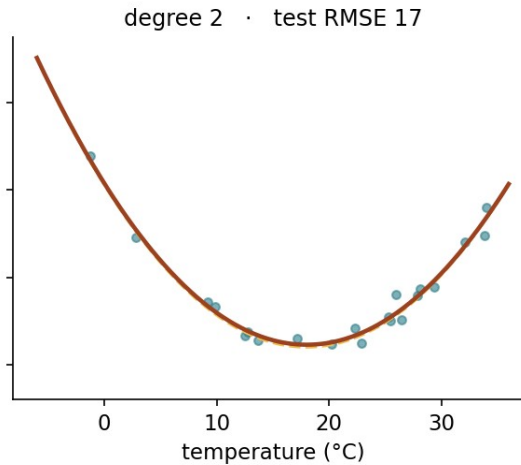
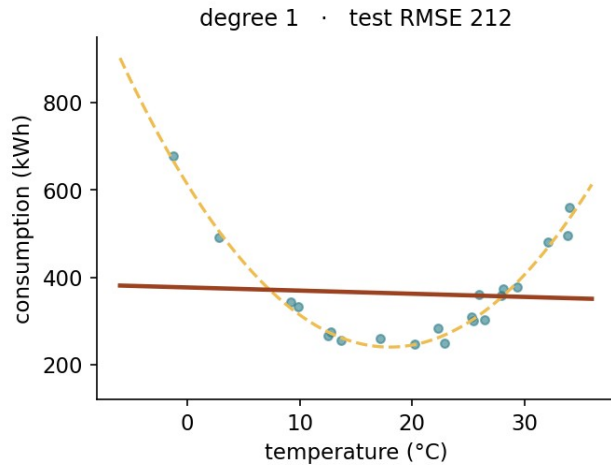
# The straight line's residuals keep the shape



# What “nothing left” looks like



# So use more flexibility?

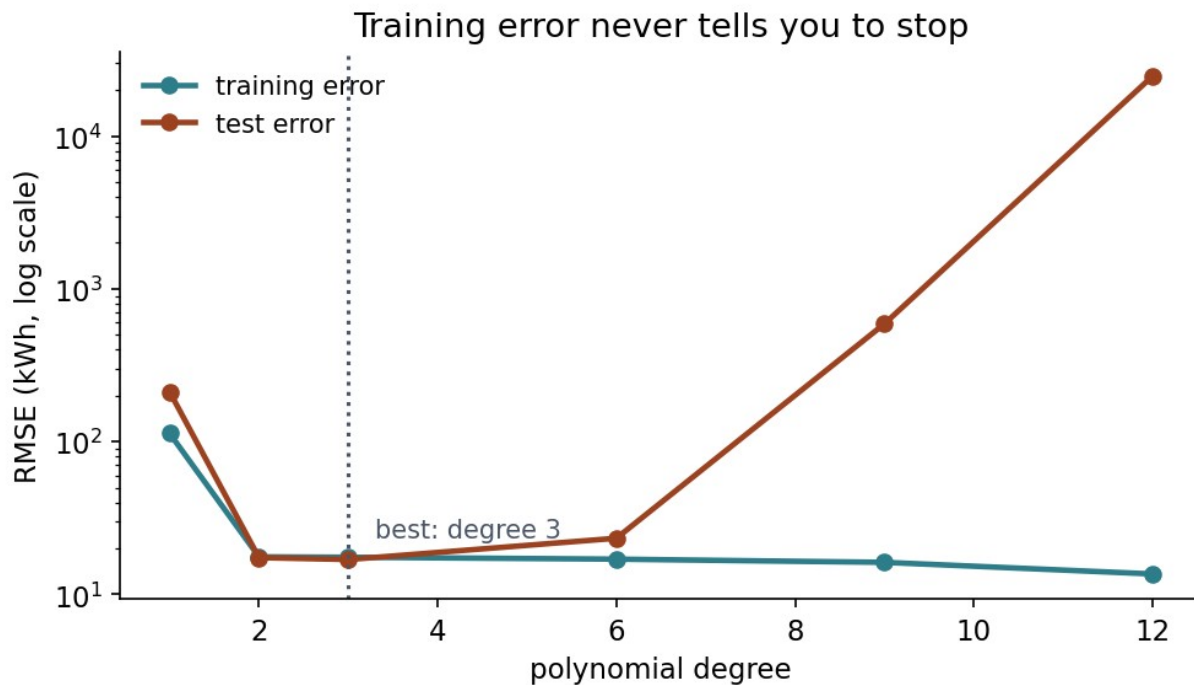


# Root mean squared error (RMSE), by degree

| Degree | Train RMSE | Test RMSE   |
|--------|------------|-------------|
| 1      | 113.3      | 212.1       |
| 3      | 17.5       | <b>16.9</b> |
| 9      | 16.2       | 590.1       |
| 12     | 13.6       | 24,655.7    |



# The gap is the overfitting



# Notebook 2, live

Fit the curve, push the degree up, and watch the test error turn.

# Why flexibility goes wrong

Largest coefficient, degree 2: **411**

Largest coefficient, degree 12: **3,097,038,010**

# Charge for size

$$J_{\text{ridge}} = \text{MSE} + \lambda \sum_j w_j^2 \quad J_{\text{lasso}} = \text{MSE} + \lambda \sum_j |w_j|$$

# Two rules that are not optional

- The **intercept is never penalised**: it is not a claim about any feature
- Features **must be scaled first**, or the penalty is arbitrary

# What a penalty buys

| Model            | Train | Test        | max  w        |
|------------------|-------|-------------|---------------|
| none             | 13.6  | 24,655.7    | 3,097,038,010 |
| $\lambda = 0.01$ | 18.0  | <b>22.8</b> | 365           |
| $\lambda = 1$    | 44.5  | 105.2       | 156           |
| $\lambda = 100$  | 96.0  | 227.6       | 6             |

# Too flexible, too rigid



Ridge has an exact solution too

$$\theta = (X^{\top} X + \lambda I)^{-1} X^{\top} y$$

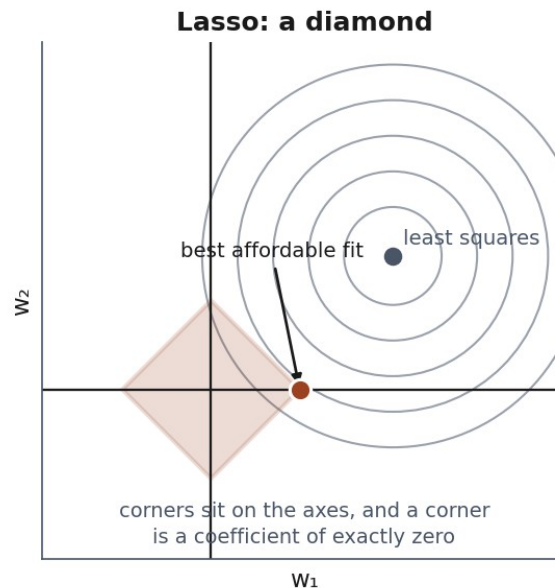
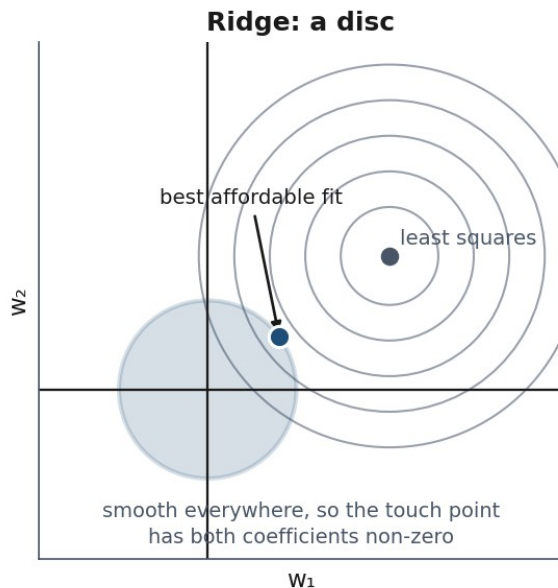


# Why that one change fixes it

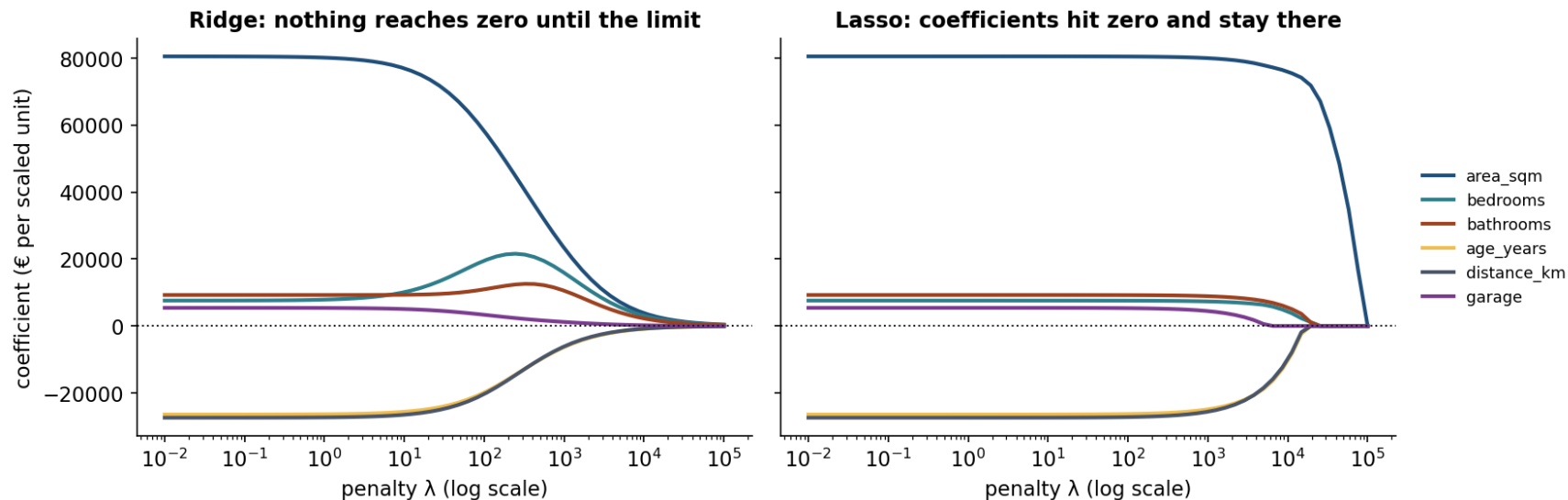
- Least squares fails when a direction costs nothing: the valley is flat
- The penalty makes **every** direction cost something
- Every eigenvalue shifts up by  $\lambda$ , so none of them can be zero

# The picture behind Ridge vs Lasso

The budget you can afford has a shape



# Ridge shrinks, Lasso selects



# The order Lasso drops features

| $\lambda$ | Features kept                    |
|-----------|----------------------------------|
| 1,000     | all six                          |
| 10,000    | five: garage dropped             |
| 20,000    | three: area, bedrooms, bathrooms |
| 40,000    | one: area_sqm                    |

# The case Ridge was invented for

area\_sqm and area\_sqft: the same measurement,  
two units.

Correlation: **0.999999**

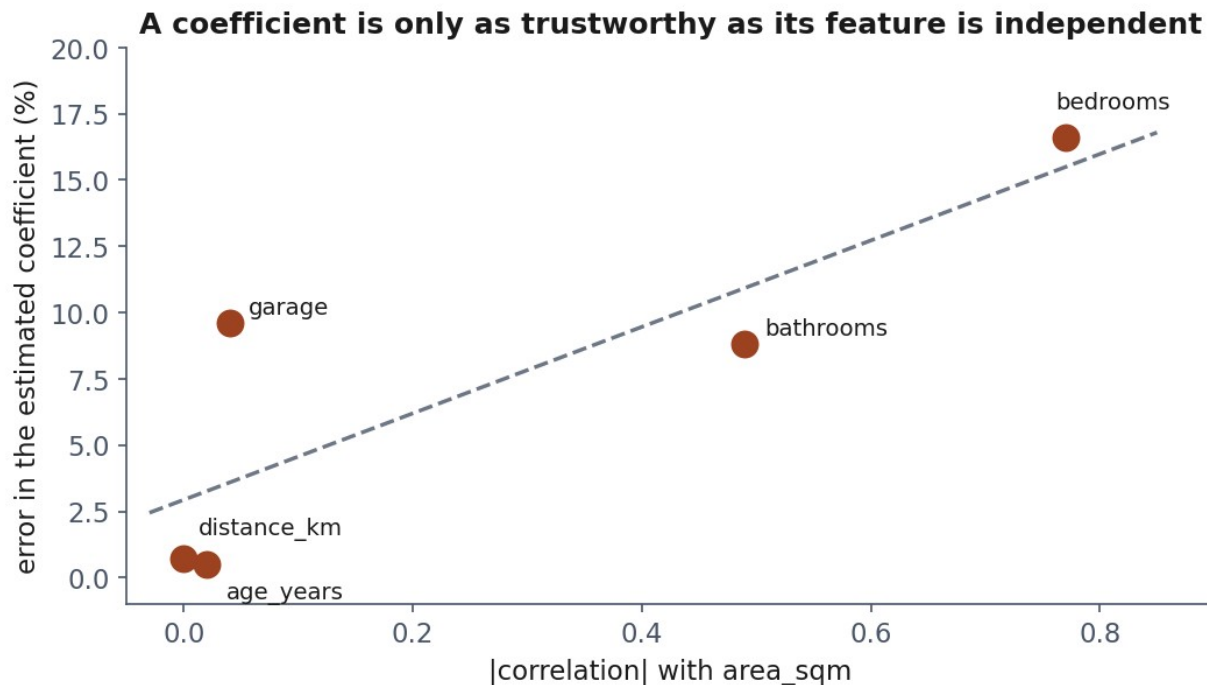
# The coefficients are noise

| Split | area_sqm | area_sqft |
|-------|----------|-----------|
| 0     | 9,261    | -640      |
| 1     | 14,535   | -1,127    |
| 2     | 45,307   | -3,989    |
| 3     | 39,061   | -3,402    |

# What a coefficient actually says

The change in the prediction for a one-unit change in that feature, **with every other feature held fixed**.

# When can you trust a coefficient?





# Notebook 3, live

Ridge on the disaster, the regularisation paths, the collinear case.

# What we did today

- A model that claims a fixed amount per unit
- An exact solution (a projection), and when it fails
- An iterative one: walk downhill, mind your stride
- Flexibility helps until it does not
- Charging for size buys generalisation, and readable coefficients
- **365** against billions: what a penalty of 0.01 costs, and buys

# Homework: we discuss it on Friday 16 October

Exercises/03\_regression.md

Fit, regularise, and **explain which coefficients  
you believe.**